

Análisis de Patrones de Diseño y Arquetipos — Sanos y Salvos

1. Introducción

El sistema **Sanos y Salvos** es una plataforma de mascotas perdidas y encontradas construida con una arquitectura de microservicios. Para garantizar su mantenibilidad, escalabilidad y eficiencia, se seleccionaron e implementaron los siguientes patrones de diseño GoF (Gang of Four) y patrones arquitectónicos, distribuidos tanto en el frontend como en el backend.

2. Patrones de Diseño Implementados

2.1 Factory Method — `ms-mascotas`

Categoría: Patrón creacional

Problema que resuelve:

El sistema maneja dos tipos de reportes: mascotas **perdidas** (`ReportePerdido`) y mascotas **encontradas** (`ReporteEncontrado`). Sin el patrón, el controlador debería contener lógica condicional (`if/else`) para instanciar el objeto correcto, acoplando la lógica de creación con la lógica de negocio.

Solución implementada:

`ReporteFactory` centraliza la creación de objetos `Reporte`. El cliente (servicio) solo invoca `ReporteFactory.crear(dto)` y la factory decide qué subclase instanciar según `dto.getTipo()`.

```
ReporteFactory
├── crear(PERDIDO)    → ReportePerdido
└── crear(ENCONTRADO) → ReporteEncontrado
```

Beneficio: El controlador y el servicio son independientes de las subclases concretas. Agregar un nuevo tipo de reporte (ej. `ReporteAvistamiento`) solo requiere modificar la factory, no los demás componentes.

Archivos:

- `ms-mascotas/src/main/java/.../factory/ReporteFactory.java`
 - `ms-mascotas/src/main/java/.../domain/Reporte.java` (clase base)
 - `ms-mascotas/src/main/java/.../domain/ReportePerdido.java`
 - `ms-mascotas/src/main/java/.../domain/ReporteEncontrado.java`
-

2.2 Observer — Arquitectura orientada a eventos (RabbitMQ)

Categoría: Patrón de comportamiento

Problema que resuelve:

Cuando se crea un reporte de mascota, el sistema necesita notificar a otros servicios: calcular coordenadas y buscar coincidencias. Si los servicios se llaman directamente, se genera acoplamiento fuerte. Un fallo en geolocalización bloquearía la creación del reporte.

Solución implementada:

`EventPublisher` en `ms-mascotas` actúa como el **sujeto observable**. Publica `ReporteNuevoEvent` en RabbitMQ. Los **observadores** son:

- `GeolocalizacionListener` en `ms-geolocalizacion`
- `CoincidenciasListener` en `ms-coincidencias`

Cada listener reacciona de forma independiente y asincrónica, sin que `ms-mascotas` los conozca.

```
ms-mascotas                RabbitMQ                Observadores
EventPublisher → reporte.nuevo → GeolocalizacionListener
                                   → CoincidenciasListener
```

Beneficio: Los servicios están completamente desacoplados. Se pueden agregar nuevos observadores sin modificar el publicador.

Archivos:

- `ms-mascotas/src/main/java/.../service/EventPublisher.java`
- `ms-mascotas/src/main/java/.../events/ReporteNuevoEvent.java`
- `ms-geolocalizacion/src/main/java/.../listener/GeolocalizacionListener.java`
- `ms-coincidencias/src/main/java/.../listener/CoincidenciasListener.java`

2.3 Proxy — BFF Service (Backend For Frontend)

Categoría: Patrón estructural

Problema que resuelve:

El frontend necesita datos de múltiples microservicios. Si el cliente hace llamadas directas a cada uno, se generan problemas de CORS, múltiples autenticaciones y un frontend fuertemente acoplado a la topología interna del backend.

Solución implementada:

El `bff-service` implementa el patrón Proxy: cada `*ProxyService` (`AuthProxyService`, `MascotasProxyService`, `GeoProxyService`, `CoincidenciasProxyService`) actúa como intermediario transparente. El frontend solo conoce un endpoint (`bff-service:8080`).

```
Frontend → BFF (Proxy) → auth-service
                                   → ms-mascotas
                                   → ms-geolocalizacion
                                   → ms-coincidencias
```

Beneficio: El frontend es independiente de la topología interna. El BFF puede transformar, agregar o cachear respuestas sin que el cliente lo note.

Archivos:

- `bff-service/src/main/java/.../service/AuthProxyService.java`
 - `bff-service/src/main/java/.../service/MascotasProxyService.java`
 - `bff-service/src/main/java/.../service/GeoProxyService.java`
 - `bff-service/src/main/java/.../service/CoincidenciasProxyService.java`
-

2.4 Strategy — ScoringService en `ms-coincidencias`

Categoría: Patrón de comportamiento

Problema que resuelve:

El algoritmo para calcular cuán parecidas son dos mascotas puede variar: algunos casos priorizan la similitud del nombre, otros la proximidad geográfica. Sin el patrón, cambiar el algoritmo requeriría modificar código disperso.

Solución implementada:

`ScoringService` encapsula la estrategia de puntuación multi-criterio. El cálculo de similitud (`FuzzyWuzzy` para texto, distancia euclidiana para coordenadas) está encapsulado y puede sustituirse sin tocar `CoincidenciaService`.

Criterios de scoring actuales:

Criterio	Herramienta	Peso
Nombre	FuzzyWuzzy	40%
Descripción	FuzzyWuzzy	30%
Ubicación	Distancia GPS	20%
Tiempo	Delta horas	10%

Archivos:

- `ms-coincidencias/src/main/java/.../service/ScoringService.java`
 - `ms-coincidencias/src/main/java/.../service/CoincidenciaService.java`
-

2.5 Context (Provider) — Frontend React

Categoría: Patrón de comportamiento (adaptado a React)

Problema que resuelve:

El estado de autenticación (usuario actual, token JWT) necesita estar disponible en múltiples componentes de la aplicación React. Sin un contexto, se haría "prop drilling" (pasar props por muchos niveles de componentes).

Solución implementada:

`AuthContext.jsx` implementa el patrón Context/Provider de React. Provee el estado de sesión a todos los

componentes de la aplicación sin pasar props explícitamente.

Archivos:

- `frontend/src/contexts/AuthContext.jsx`
- `frontend/src/components/PrivateRoute.jsx`

2.6 Circuit Breaker — `bff-service` con Resilience4j

Categoría: Patrón de resiliencia (arquitectónico)

Problema que resuelve:

Si un microservicio cae o responde lento, las solicitudes del BFF quedarían bloqueadas indefinidamente, causando una cascada de fallos que afecta a toda la aplicación.

Solución implementada:

Resilience4j implementa el Circuit Breaker en `bff-service`. Cuando las llamadas a un microservicio superan el umbral de fallos, el circuito "abre" y retorna respuestas de fallback inmediatamente, protegiendo el sistema.

Archivos:

- `bff-service/pom.xml` (dependencia resilience4j)
- `bff-service/src/main/java/.../config/RestTemplateConfig.java`

3. Justificación de la selección

Patrón	Problema central que resuelve	Alternativa descartada
Factory Method	Creación de múltiples tipos de reporte	<code>if/else</code> acoplado en el servicio
Observer	Comunicación asincrónica entre servicios	Llamadas REST directas (acoplamiento fuerte)
Proxy (BFF)	Punto de entrada único para el frontend	Frontend llamando a cada microservicio
Strategy	Algoritmo de scoring intercambiable	Lógica hardcodeada en el servicio
Context (React)	Estado global de autenticación en UI	Prop drilling por toda la jerarquía
Circuit Breaker	Tolerancia a fallos en llamadas HTTP	Sin resiliencia (cascada de fallos)

4. Arquetipos Maven utilizados

4.1 ¿Qué es un arquetipo Maven?

Un arquetipo Maven es una **plantilla de proyecto** que genera la estructura inicial de un proyecto Java con todas las dependencias y configuraciones base. Equivale a un "scaffolding" para proyectos Java/Spring.

4.2 Arquetipo utilizado: Spring Boot Initializr

Todos los microservicios del backend se generaron usando el arquetipo **Spring Boot** a través de [Spring Initializr](#), que corresponde al arquetipo Maven:

```
groupId:    org.springframework.boot
artifactId: spring-boot-starter-parent
version:    3.2.5
```

Este arquetipo provee:

- Estructura de directorios estándar Maven ([src/main/java](#), [src/test/java](#))
- Plugin de Spring Boot para empaquetar como JAR ejecutable
- Gestión de dependencias transitivas
- Plugin de JaCoCo para cobertura de tests

4.3 Arquetipo personalizado del proyecto

Para estandarizar la creación de nuevos microservicios dentro del ecosistema **Sanos y Salvos**, se definió un arquetipo Maven personalizado (ver directorio [/maven-archetypes/](#)). Este arquetipo incluye:

- Estructura base de paquetes [com.sanosysalvos.*](#)
- Dependencias comunes preconfiguradas (JWT, JaCoCo, Lombok)
- Configuración de [JwtAuthFilter](#) lista para usar
- Configuración de RabbitMQ estándar
- Tests de integración base

Ver detalles en [/maven-archetypes/README.md](#).

5. Resumen de componentes y patrones

Sistema Sanos y Salvos	
├─ frontend/	→ Context Pattern, Proxy Pattern (services)
├─ auth-service/	→ Chain of Responsibility, Repository Pattern
├─ bff-service/	→ Proxy Pattern, Circuit Breaker, Facade Pattern
├─ ms-mascotas/	→ Factory Method, Observer (Publisher)
├─ ms-geolocalizacion/	→ Observer (Listener), Strategy (Clustering)
└─ ms-coincidencias/	→ Observer (Listener), Strategy (Scoring)